

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Ciencias Exactas e Ingenierías

Ingeniería en Informática

Fundamentos de Bases de Datos

ROOMLY

Sistema de gestión hotelera

Proyecto

Equipo Azul

Melanye Guadalupe Ramos Muñoz

Alicia Lizbeth Pérez Gómez

Jesús Armando Juárez Cabrera

Leonardo David Martínez González

Guadalajara, Jalisco. 26 de agosto de 2026

Índice

1. Introducción	3
2. Planteamiento del problema	3
3. Justificación	3
4. Objetivos	4
5. Alcance del proyecto	4
6. Arquitectura y herramientas	5
7. Plan de trabajo	5
8. Organización del equipo	6
9. Modelo de datos	7
9.1 Reglas que definimos	7
9.2 Diagrama entidad-relación	8
9.3 Diccionario de datos	9
9.4 Por qué el modelo quedó así	12
10. Cronograma	13
11. Referencias	13

1. Introducción

Un hotel maneja todos los días la misma información: quién se hospeda, qué habitaciones tiene y cuáles están ocupadas en qué fechas. Cuando eso se lleva en una libreta o en hojas de cálculo sueltas, tarde o temprano pasa lo mismo, que se le asigna la misma habitación a dos personas en las mismas noches.

En este proyecto desarrollamos ROOMLY, una base de datos relacional para administrar los clientes, las habitaciones y las reservaciones de un hotel, junto con una aplicación para operarla. El peso del trabajo está en el modelo de datos. La aplicación sirve para comprobar que ese modelo responde bien a lo que el hotel necesita preguntar todos los días.

2. Planteamiento del problema

Un hotel que lleva su información de forma desorganizada tiene por lo menos cuatro problemas:

- No sabe con certeza si una habitación está libre en cierto rango de fechas. La disponibilidad no es un dato fijo, cambia según las reservaciones que existan en ese momento.
- Repite la información de los clientes. Un huésped que regresa se vuelve a capturar, a veces con el nombre escrito distinto, y ya no se puede saber cuántas veces se ha hospedado.
- Pierde el historial. Si se modifica o se cancela una reservación, el dato anterior se sobrescribe y no queda registro de lo que se había cobrado.
- No se puede consultar nada útil. Preguntas como cuál tipo de habitación se ocupa más, o cuánta ocupación hubo el mes pasado, se quedan sin respuesta.

3. Justificación

Una base de datos relacional resuelve estos problemas desde el diseño. Al separar la información en tablas relacionadas y ponerle restricciones, el propio gestor impide que se guarden datos inconsistentes. Por ejemplo, no deja registrar una reservación de un cliente que no existe, sin que nadie tenga que acordarse de revisarlo.

Escogimos un hotel porque es un caso donde aparecen de forma natural los dos tipos de relación que vimos en clase. Hay relaciones de uno a muchos, como la de un cliente con sus reservaciones, y hay una de muchos a muchos entre las reservaciones y las habitaciones, que obliga a usar una tabla intermedia. Eso nos permitía aplicar llaves primarias, llaves foráneas y normalización sobre algo concreto y no sobre un ejemplo inventado.

4. Objetivos

4.1 Objetivo general

Desarrollar una base de datos relacional que permita administrar de manera organizada los clientes, las habitaciones y las reservaciones de un hotel, facilitando el control de la disponibilidad y las operaciones relacionadas con las reservaciones.

4.2 Objetivos específicos

1. Diseñar el modelo entidad-relación, definiendo las entidades, sus atributos, las llaves primarias y foráneas, y el tipo de cada relación.
2. Normalizar el modelo hasta la tercera forma normal.
3. Implementar el esquema en PostgreSQL con sus restricciones de integridad.
4. Cargar datos de prueba que cubran los casos difíciles, como reservaciones que se traslapan y reservaciones canceladas.
5. Escribir las consultas que necesita el hotel: registrar y consultar clientes, dar de alta, modificar y cancelar reservaciones, y consultar qué habitaciones están libres en un rango de fechas.
6. Construir una aplicación web de tres capas que use la base de datos.
7. Documentar el proyecto con el diccionario de datos y el diagrama.

5. Alcance del proyecto

El sistema va a cubrir el registro y la consulta de clientes, el catálogo de habitaciones, el alta, la modificación, la consulta y la cancelación de reservaciones, la consulta de disponibilidad por fechas y el registro de los pagos de cada reservación. Todo eso se opera desde una aplicación web.

Hay cosas que decidimos dejar fuera para que el proyecto sea realista en el tiempo que tenemos:

- Cobros reales o conexión con una pasarela de pagos.
- Facturación electrónica.
- Nómina, control de personal e inventarios.
- Aplicación para celular.
- Conexión con agencias de viaje en línea.
- Historial de cambios de una reservación. Solo guardamos el estado en el que está actualmente.

6. Arquitectura y herramientas

El sistema está pensado en tres capas. El navegador no habla directo con la base de datos, siempre pasa por el servidor de aplicación, que es el único que tiene permitido consultar el gestor. Así la base queda protegida y la lógica del negocio vive en un solo lugar.

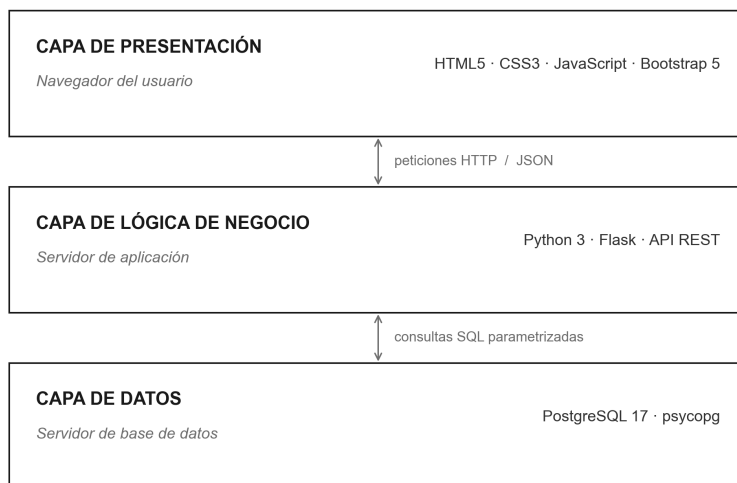


Figura 1. Arquitectura de tres capas del sistema.

Tabla 1. Herramientas elegidas para cada capa.

Capa	Herramienta	Por qué la elegimos
Presentación	HTML, CSS, JavaScript y Bootstrap	No necesita compilarse ni instalar un framework, así el tiempo se va al modelo de datos y no a pelearse con la interfaz.
Lógica de negocio	Python con Flask	Es el framework más pequeño que conocemos y el equipo ya había trabajado con Python.
Datos	PostgreSQL 17	Es el gestor que usamos en clase y tiene soporte completo de restricciones y de tipos de fecha, que es lo que más ocupamos aquí.
Conexión	psycopg	Es el conector de PostgreSQL para Python. Permite consultas con parámetros, lo que evita la inyección de SQL.

Además usamos pgAdmin para administrar la base, GitHub para el control de versiones y Canva para la presentación.

7. Plan de trabajo

Trabajamos por etapas, y cada una tenía que dejar algo revisable antes de pasar a la siguiente:

1. Análisis. Definir qué hace el hotel y con qué reglas.
2. Diseño. Modelo entidad-relación, normalización y diccionario de datos.
3. Implementación. Script de creación de tablas, datos de prueba, consultas y desarrollo de la aplicación.
4. Pruebas y documentación. Probar el modelo con casos difíciles y cerrar la documentación.

El código está en GitHub, en el repositorio `sistema-gestion-hotel`, para que quede registro de cuándo se hizo cada cosa.

8. Organización del equipo

No nos repartimos el proyecto por módulos. Las etapas las trabajamos en sesiones conjuntas, de manera que todos participáramos en el análisis, en el diseño del modelo y en la documentación.

Lo hicimos así a propósito. El modelo de datos es la parte más importante del proyecto y todos tenemos que poder explicarlo, y eso no pasa si cada quien conoce nada más su pedazo.

Tabla 2. Integrantes del equipo y sus cuentas de GitHub.

Alumno	Usuario de GitHub
Melanye Guadalupe Ramos Muñoz	melirm05
Alicia Lizbeth Pérez Gómez	aliperezg
Jesús Armando Juárez Cabrera	jesusjuarez6280-droid
Leonardo David Martínez González	Jorjais-craft

9. Modelo de datos

9.1 Reglas que definimos

Antes de dibujar nada tuvimos que ponernos de acuerdo en cómo funciona el hotel, porque cada respuesta cambia la forma de las tablas. Estas son las siete reglas que acordamos y lo que provocó cada una:

Tabla 3. Reglas del negocio y su efecto en el modelo.

Regla	Qué provocó en el modelo
Una reservación puede incluir varias habitaciones, y una habitación entra en muchas reservaciones a lo largo del tiempo.	Relación de muchos a muchos entre reservación y habitación, resuelta con la tabla intermedia <code>reservacion_habitacion</code> .
La disponibilidad no se guarda, se deduce de las reservaciones que hay.	No existe ninguna columna <code>disponible</code> . Se calcula comparando fechas.
Las habitaciones se agrupan por tipo, y el precio de lista es del tipo, no de la habitación.	Una tabla <code>tipo_habitacion</code> aparte de <code>habitacion</code> .
Una reservación tiene un solo estado a la vez.	Columna <code>estado</code> limitada con una restricción <code>CHECK</code> .
Una reservación puede tener varios pagos, por ejemplo un anticipo y el resto.	Relación de uno a muchos de reservación a pago.
Toda reservación la registra un empleado en recepción.	Relación de uno a muchos de empleado a reservación.
La reservación guarda el precio que de verdad se cobró, aunque después cambie la lista.	Columna <code>precio_noche</code> dentro de <code>reservacion_habitacion</code> .

9.2 Diagrama entidad-relación

De esas reglas salieron siete tablas. En el diagrama las llaves primarias van en negritas y las foráneas en cursivas.

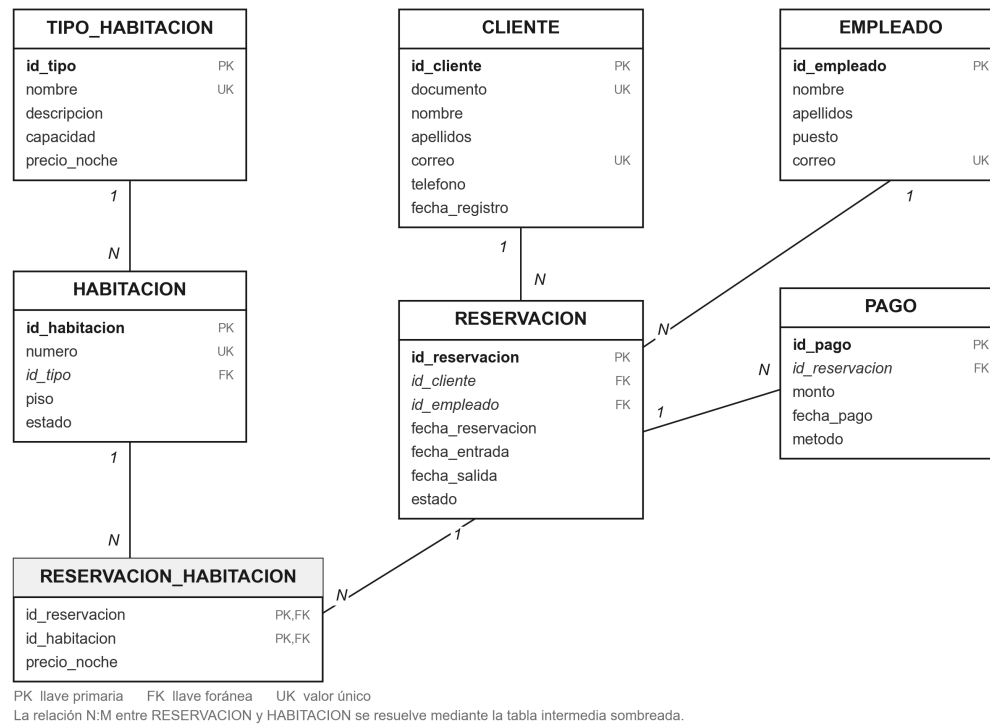


Figura 2. Modelo entidad-relación de ROOMLY.

Tabla 4. Cardinalidad de cada relación.

Relación	Tipo	Cómo se implementa
cliente y reservación	1 a N	Llave foránea <i>id_cliente</i> en reservacion
empleado y reservación	1 a N	Llave foránea <i>id_empleado</i> en reservacion
tipo de habitación y habitación	1 a N	Llave foránea <i>id_tipo</i> en habitacion
reservación y pago	1 a N	Llave foránea <i>id_reservacion</i> en pago
reservación y habitación	N a M	Tabla intermedia <i>reservacion_habitacion</i>

9.3 Diccionario de datos

Tabla 5. Tabla `cliente`. Huéspedes registrados en el hotel.

Columna	Tipo	Restricciones	Descripción
id_cliente	SERIAL	Llave primaria	Identificador interno
documento	VARCHAR(20)	NOT NULL, UNIQUE	Identificación oficial
nombre	VARCHAR(60)	NOT NULL	Nombre del huésped
apellidos	VARCHAR(80)	NOT NULL	Apellidos del huésped
correo	VARCHAR(120)	NOT NULL, UNIQUE	Correo de contacto
telefono	VARCHAR(20)	Opcional	Teléfono de contacto
fecha_registro	DATE	NOT NULL	Fecha en que se dio de alta

Tabla 6. Tabla `empleado`. Personal que opera el sistema.

Columna	Tipo	Restricciones	Descripción
id_empleado	SERIAL	Llave primaria	Identificador interno
nombre	VARCHAR(60)	NOT NULL	Nombre del empleado
apellidos	VARCHAR(80)	NOT NULL	Apellidos del empleado
puesto	VARCHAR(40)	NOT NULL	Recepción, gerencia, etcétera
correo	VARCHAR(120)	NOT NULL, UNIQUE	Correo del trabajo

Tabla 7. Tabla `tipo_habitacion`. Catálogo de categorías y su precio.

Columna	Tipo	Restricciones	Descripción
id_tipo	SERIAL	Llave primaria	Identificador interno
nombre	VARCHAR(40)	NOT NULL, UNIQUE	Sencilla, doble, suite, familiar
descripcion	VARCHAR(200)	Opcional	Qué incluye la habitación
capacidad	SMALLINT	NOT NULL, mayor a cero	Cuántas personas caben
precio_noche	NUMERIC(10,2)	NOT NULL, mayor a cero	Precio de lista por noche

Tabla 8. Tabla `habitacion`. Habitaciones físicas del hotel.

Columna	Tipo	Restricciones	Descripción
<code>id_habitacion</code>	SERIAL	Llave primaria	Identificador interno
<code>numero</code>	VARCHAR(10)	NOT NULL, UNIQUE	Número visible, como 101 o 202
<code>id_tipo</code>	INTEGER	Llave foránea	Categoría a la que pertenece
<code>piso</code>	SMALLINT	NOT NULL, mayor a cero	Piso donde está
<code>estado</code>	VARCHAR(15)	CHECK	Operativa, en mantenimiento o dada de baja

Tabla 9. Tabla `reservacion`. Reservaciones hechas por los clientes.

Columna	Tipo	Restricciones	Descripción
<code>id_reservacion</code>	SERIAL	Llave primaria	Identificador interno
<code>id_cliente</code>	INTEGER	Llave foránea	Huésped que reserva
<code>id_empleado</code>	INTEGER	Llave foránea	Quien registró la reservación
<code>fecha_reservacion</code>	DATE	NOT NULL	Cuándo se hizo la reserva
<code>fecha_entrada</code>	DATE	NOT NULL	Día de llegada
<code>fecha_salida</code>	DATE	NOT NULL, CHECK	Día de salida, siempre posterior a la entrada
<code>estado</code>	VARCHAR(12)	CHECK	Pendiente, confirmada, en curso, finalizada o cancelada

Tabla 10. Tabla `reservacion_habitacion`. Qué habitaciones lleva cada reservación.

Columna	Tipo	Restricciones	Descripción
<code>id_reservacion</code>	INTEGER	Parte de la llave primaria y llave foránea	Reservación
<code>id_habitacion</code>	INTEGER	Parte de la llave primaria y llave foránea	Habitación asignada
<code>precio_noche</code>	NUMERIC(10,2)	NOT NULL, mayor a cero	Precio que se cobró, guardado al momento de reservar

Tabla 11. Tabla pago. Abonos de cada reservación.

Columna	Tipo	Restricciones	Descripción
id_pago	SERIAL	Llave primaria	Identificador interno
id_reservacion	INTEGER	Llave foránea	Reservación que se está abonando
monto	NUMERIC(10,2)	NOT NULL, mayor a cero	Cuánto se pagó
fecha_pago	DATE	NOT NULL	Cuándo se pagó
metodo	VARCHAR(15)	CHECK	Efectivo, tarjeta o transferencia

9.4 Por qué el modelo quedó así

Hay tres decisiones que no son obvias y que vale la pena explicar, porque son las que más nos costó discutir.

No hay una columna de disponibilidad

Lo primero que se nos ocurrió fue ponerle a cada habitación una columna que dijera si está disponible o no. No funciona. Una habitación puede estar libre el 3 de septiembre y ocupada el 4, así que la disponibilidad no es una característica de la habitación, depende de la fecha por la que se pregunte. Si la guardáramos habría que recalcularla todos los días y se desincroniza al primer error de captura.

En lugar de eso la deducimos. Una habitación está libre en un rango si no hay ninguna reservación sin cancelar que se le encime. Dos rangos se enciman cuando se cumplen las dos condiciones al mismo tiempo:

```
fecha_entrada de la reservación existente < fecha_salida de la solicitada
fecha_salida de la reservación existente > fecha_entrada de la solicitada
```

La columna `estado` de la habitación es otra cosa y no hay que confundirla. Esa dice si la habitación está fuera de servicio por mantenimiento, que sí es una característica de la habitación y no depende del calendario.

El precio está en dos lugares y no es redundancia

El `precio_noche` de `tipo_habitacion` es el precio de lista de hoy. El `precio_noche` de `reservacion_habitacion` es lo que se le cobró a un huésped en específico cuando reservó. Son dos datos distintos y los dos se necesitan.

Si nada más guardáramos el de lista, el día que el hotel suba tarifas en temporada alta cambiarían solos los importes de todas las reservaciones viejas, y las cuentas del mes pasado dejarían de cuadrar.

El tipo de habitación es una tabla aparte

Si la descripción, la capacidad y el precio estuvieran dentro de `habitacion`, se repetirían iguales en todas las habitaciones de la misma categoría. Cambiar el precio de las suites obligaría a modificar renglón por renglón.

Aparte de la repetición, ahí hay una dependencia transitiva: el identificador de la habitación determina su tipo, y el tipo determina el precio. La tercera forma normal no permite eso, y separar la tabla lo resuelve.

10. Cronograma

Tabla 12. Etapas del proyecto durante agosto de 2026.

Etapas	Fechas	Estado
Definición del tema y formación del equipo	18 y 19 de agosto	Terminada
Investigación y planteamiento del problema	20 y 21 de agosto	Terminada
Definición de las reglas del negocio y los objetivos	22 de agosto	Terminada
Diseño del modelo, normalización y diccionario	23 y 24 de agosto	Terminada
Documento y presentación	25 de agosto	Terminada
Entrega	26 de agosto	En curso

Las siguientes etapas, ya de implementación, quedaron programadas para septiembre:

Tabla 13. Etapas de implementación durante septiembre de 2026.

Etapas	Fechas
Crear el esquema en PostgreSQL y cargar los datos de prueba	27 de agosto al 2 de septiembre
Desarrollo de la API en Flask	3 al 10 de septiembre
Desarrollo de la interfaz web	11 al 18 de septiembre
Pruebas y documentación final	19 al 25 de septiembre

11. Referencias

Elmasri, R. y Navathe, S. B. *Fundamentos de sistemas de bases de datos*. Pearson Educación.

Silberschatz, A., Korth, H. F. y Sudarshan, S. *Fundamentos de bases de datos*. McGraw-Hill.

Date, C. J. *Introducción a los sistemas de bases de datos*. Pearson Educación.

The PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*.
<https://www.postgresql.org/docs/17/>